

OS PRACTICAL 7 & 8

7. Memory Segments using malloc()

Code:

```
GNU nano 8.7.1 practical7.c
#include <stdio.h>
#include <stdlib.h>
int global=100;
static int s=50;
int main()
{
    int stack=10;
    int *heap=(int*)malloc(sizeof(int));
    printf("Code Segment : %p\n", main);
    printf("Global Segment : %p\n", &global);
    printf("Static Segment : %p\n", &s);
    printf("Heap Segment : %p\n", heap);
    printf("Stack Segment : %p\n", &stack);
    free(heap);
    return 0;
}
```

Output:

```
gayathri@Gayathri: ~$ nano practical7.c
gayathri@Gayathri: ~$ gcc practical7.c -o practical7
gayathri@Gayathri: ~$ ./practical7
Code Segment : 0x63a1451ec1a9
Global Segment : 0x63a1451ef010
Static Segment : 0x63a1451ef014
Heap Segment : 0x63a160866010
Stack Segment : 0x7ffde1b94d4c
gayathri@Gayathri: ~$
```

8. Dynamic Memory Allocation using malloc(), calloc(), realloc() and free()

Code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int *a,*b;
    a=(int*)malloc(5*sizeof(int));
    b=(int*)calloc(5,sizeof(int));
    a=(int*)realloc(a,10*sizeof(int));
    free(a);
    free(b);
    return 0;
}
```

Output:

```
gayathri@Gayathri: ~  
gayathri@Gayathri:~$ nano memory.c  
gayathri@Gayathri:~$ gcc memory.c -o memory  
gayathri@Gayathri:~$ ./memory  
Code address   : 0x5bb89aa3e1a9  
Data address   : 0x5bb89aa41010  
BSS address    : 0x5bb89aa41018  
Heap address   : 0x5bb89f706010  
Stack address  : 0x7ffff835a3ec  
gayathri@Gayathri:~$ valgrind --leak-check=full --show-leak-kinds=all ./memo  
ry  
==715== Memcheck, a memory error detector  
==715== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.  
==715== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info  
==715== Command: ./memory  
==715==  
Code address   : 0x40011a9  
Data address   : 0x4004010  
BSS address    : 0x4004018  
Heap address   : 0x4a90040  
Stack address  : 0x1ffeffffbfc  
==715==  
==715== HEAP SUMMARY:  
==715==    in use at exit: 0 bytes in 0 blocks  
==715==   total heap usage: 2 allocs, 2 frees, 1,028 bytes allocated  
==715==  
==715== All heap blocks were freed -- no leaks are possible  
==715==  
==715== For lists of detected and suppressed errors, rerun with: -s  
==715== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)  
gayathri@Gayathri:~$ |
```

8b. Copy-on-Write:

Code:

```
GNU nano 8.7.1 cow.c *  
#include <stdio.h>  
#include <unistd.h>  
int main()  
{  
    int x=100;  
    if(fork()==0)  
    {  
        printf("Child Before=%d\n",x);  
        x=500;  
        printf("Child After=%d\n",x);  
    }  
    else  
    {  
        sleep(2);  
        printf("Parent=%d\n",x);  
    }  
    return 0;  
}
```

Output:

```
gayathri@Gayathri:~$ nano cow.c
gayathri@Gayathri:~$ gcc cow.c -o cow
gayathri@Gayathri:~$ ./cow
Child Before=100
Child After=500
Parent=100
gayathri@Gayathri:~$ valgrind --leak-check=full ./cow
==770== Memcheck, a memory error detector
==770== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==770== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==770== Command: ./cow
==770==
Child Before=100
Child After=500
==771==
==771== HEAP SUMMARY:
==771==     in use at exit: 0 bytes in 0 blocks
==771==   total heap usage: 1 allocs, 1 frees, 1,024 bytes allocated
==771==
==771== All heap blocks were freed -- no leaks are possible
==771==
==771== For lists of detected and suppressed errors, rerun with: -s
==771== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
Parent=100
==770==
==770== HEAP SUMMARY:
==770==     in use at exit: 0 bytes in 0 blocks
==770==   total heap usage: 1 allocs, 1 frees, 1,024 bytes allocated
==770==
==770== All heap blocks were freed -- no leaks are possible
==770==
==770== For lists of detected and suppressed errors, rerun with: -s
==770== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```